

1a)

```
SELECT f.nome_fabricante,  
  
SUM(p.preco * v.quantidade) AS receita_total  
  
FROM fabricante f  
  
JOIN produto p ON p.id_fabricante = f.id_fabricante  
  
JOIN venda v ON v.id_produto = p.id_produto  
  
GROUP BY f.nome_fabricante  
  
HAVING SUM(p.preco * v.quantidade) >  
  
    ( SELECT AVG(receita)  
  
      FROM ( SELECT SUM(p2.preco * v2.quantidade) AS receita  
  
            FROM fabricante f2  
  
            JOIN produto p2 ON p2.id_fabricante = f2.id_fabricante  
  
            JOIN venda v2 ON v2.id_produto = p2.id_produto  
  
            GROUP BY f2.id_fabricante ) sub );
```

1b) A subquery no HAVING é importante para calcular a média das receitas totais de todos os fabricantes. Ele precisa ser colocado em uma subconsulta porque não ser feito agregação sobre agregação no SQL diretamente no mesmo nível da consulta principal. A subquery agrupa as receitas por fabricante e calcula a média global, permitindo que o HAVING compare o "total deste fabricante" com a "média de todos".

2a) O LEFT JOIN com SUM e AVG calcula o desempenho de vendas de cada loja, assegurando que todas apareçam no relatório, até as que não possuem vendas. LEFT JOIN garante que a loja seja listada mesmo sem correspondência na tabela de vendas. O SUM volta com NULL e o AVG volta com NULL para lojas sem vendas, permitindo relatórios completos do catálogo.

2b) Sim, dado que a conversão é recomendada para relatórios de desempenho recorrentes e pesados. O VIEW comum recalcula as agregações a cada acesso. O

MATERIALIZED VIEW guarda o resultado físico da consulta, melhorando drasticamente o tempo de resposta grandes volumes.

2c)

```
CREATE OR REPLACE VIEW vw_loja_desempenho AS

SELECT l.id_loja,

       l.nome_loja,

       SUM(v.quantidade) AS total_vendido,

       AVG(p.preco) AS media_preco_vendido,

       COUNT(DISTINCT p.id_produto) AS quantidade_itens_diferentes

FROM loja l

LEFT JOIN venda v ON v.id_loja = l.id_loja

LEFT JOIN produto p ON p.id_produto = v.id_produto

GROUP BY l.id_loja, l.nome_loja;
```

3a) A procedure aceita valores negativos e não faz verificação para determinar se cliente já foi inscrito em tal evento.

3b)

```
CREATE OR REPLACE PROCEDURE sp_registra_inscricao(

    p_id_evento INT,

    p_id_cliente INT,

    p_equipe TEXT,

    p_taxa NUMERIC

)

LANGUAGE plpgsql

AS $$

BEGIN
```

```

-- 1. Validação de taxa negativa

IF p_taxa < 0 THEN

    RAISE EXCEPTION 'Taxa não pode ser negativa. Valor inserido: %', p_taxa;

END IF;


-- 2. Impedir inscrição duplicada

IF EXISTS (

    SELECT 1

    FROM inscricao

    WHERE id_evento = p_id_evento

    AND id_cliente = p_id_cliente

) THEN

    RAISE EXCEPTION 'O cliente com ID % já está inscrito no evento com ID %',
p_id_cliente, p_id_evento;

END IF;


INSERT INTO inscricao(id_evento, id_cliente, equipe, taxa_inscricao)

VALUES (p_id_evento, p_id_cliente, p_equipe, p_taxa);


RAISE NOTICE 'Inscrição registrada com sucesso para o cliente % no evento %.',
p_id_cliente, p_id_evento;

END;

$$;

```

4a) A query identifica clientes cujo gasto pessoal total é superior à soma total de vendas de todas as lojas "Paulista". Compara o gasto individual de um cliente com o histórico completo de uma loja individual.

4b) O HAVING é obrigatório porque a comparação é feita sobre um valor agregado (SUM). A cláusula WHERE filtra linhas antes do agrupamento e não pode acessar resultados de cálculos agregados.

4c) Identificação de clientes "VIP" ou "Baleias" para campanhas de fidelização, atendimento personalizado ou benefícios premium, dado que o consumo deles supera o faturamento de uma filial inteira.

5a) A procedure adiciona o sufixo "(Atenção)" a produtos com baixa performance. Ela depende da view vw_media_produto porque esta já encapsula a lógica complicada de calcular a média de vendas, tornando o código mais simples.

5b) Executar a procedure várias vezes pode causar a duplicação do sufixo, inflando e poluindo os dados e estragando a consistência do banco.

5c) Adicionar uma verificação para garantir que o texto só seja adicionado se ainda não estiver presente.

```
CREATE OR REPLACE PROCEDURE sp_flag_produtos_baixa_media()
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    UPDATE produto
```

```
    SET nome = nome || ' (Atenção)'
```

```
    WHERE id_produto IN (
```

```
        SELECT id_produto FROM vw_media_produto WHERE media_por_loja < 2
    )
```

```
    AND nome NOT ILIKE '%(Atenção)%'; -- Impede a duplicação
```

```
END;
```

```
$$;
```